Plano e Execução de Testes — Frontend

Sistema de Gerenciamento de Biblioteca (Bibton)

Disciplina: Engenharia de Software II **Equipe 03 — Projeto:** Gerenciamento de Biblioteca **Camada testada:** Frontend (aplicação web React) **Data:** 31/05/2026

> Este documento cobre a **camada de frontend**. A camada de backend (microsserviços) foi documentada em trabalho separado. Os testes foram executados com o frontend e o backend rodando **localmente** (ver Anexo A — Ambiente de Testes).

PARTE 1 — Descrição da Aplicação

- **Nome da aplicação:** Bibton — Sistema de Gerenciamento de Biblioteca
- **Objetivo do sistema:** Permitir que bibliotecários e membros gerenciem o acervo e a operação de uma biblioteca: cadastro de usuários, catálogo de livros, empréstimos, devoluções, reservas e controle de multas por atraso.
- **Tecnologias utilizadas (frontend):**
  - React 18 + Vite 5
  - React Router DOM 6 (navegação SPA)
  - React Hook Form + Zod (formulários e validação)
  - Axios (consumo da API via API Gateway)
  - Tailwind CSS (estilização e tema claro/escuro)
  - React Toastify (notificações), Lucide React (ícones), GSAP (animações)
- **Arquitetura básica:**
  - SPA (Single Page Application) que consome uma API REST exposta por um **API Gateway**.
  - O backend segue arquitetura de **microsserviços** (usuários, livros, empréstimos, multas, notificações), cada um com seu próprio banco PostgreSQL.
  - Camadas do frontend: `pages/` (telas), `components/forms/` (formulários), `contexts/` (estado global de autenticação e tema), `config/api.jsx` (cliente HTTP).
- **Funcionalidades principais:**
  1. Dashboard com indicadores gerais
  2. CRUD de Usuários
  3. CRUD de Livros (catálogo)
  4. Registro e devolução de Empréstimos
  5. Reservas de livros (criar/cancelar)
  6. Multas (visualizar/pagar)
  7. Tema claro/escuro e responsividade

PARTE 2 — Levantamento das Funcionalidades Críticas

> Pergunta-guia: *"Se essa funcionalidade falhar, o sistema quebra ou gera grande impacto?"*

Criticidade	Funcionalidade	Justificativa
Alta	Autenticação / controle de acesso	Protege o sistema; falha expõe todos os dados.
Alta	Comunicação com a API (Gateway)	Sem ela, nenhuma tela carrega ou persiste dados.
Alta	Cadastro/edição de Livros e Usuários	Persistência no banco; base de todo o sistema.

Criticidade	Funcionalidade	Justificativa
Alta	Empréstimo e Devolução	Fluxo central de negócio da biblioteca.
Média	Reservas	Funcionalidade de apoio com solução alternativa.
Média	Multas	Importante, mas não bloqueia o uso geral.
Média	Pesquisa e filtros	Afeta usabilidade, não a integridade dos dados.
Baixa	Tema escuro	Apenas visual.
Baixa	Animações (GSAP)	Apenas estético.

PARTE 3 — Casos de Teste

> 20 casos cobrindo entradas **válidas** e **inválidas**, testes funcionais, de validação, de segurança e de usabilidade.

ID	Funcionalidade	Entrada	Resultado Esperado	Tipo
CT01	Dashboard	Acessar `/`	Carrega indicadores e dados vindos da API	Funcional
CT02	Cadastro de Usuário	Nome + e-mail válido + cargo	Usuário criado e listado	Funcional
CT03	Cadastro de Usuário	E-mail inválido (`abc`)	Mensagem "Email inválido"; cadastro impedido	Validação
CT04	Cadastro de Usuário	Nome em branco	Mensagem "Nome é obrigatório"; cadastro impedido	Validação
CT05	Edição de Usuário	Alterar nome de usuário existente	Dados atualizados na lista	Funcional
CT06	Exclusão de Usuário	Confirmar exclusão	Usuário removido após confirmação	Funcional
CT07	Cadastro de Livro	Título + autor + nº de cópias	Livro criado e listado no catálogo	Funcional
CT08	Cadastro de Livro	Título em branco	Mensagem "Título é obrigatório"; impedido	Validação
CT09	Cadastro de Livro	Cópias = `-5`	Mensagem "Não pode ser negativo"; impedido	Validação
CT10	Empréstimo	Selecionar usuário + livro disponível	Empréstimo registrado; cópias decrementadas	Funcional
CT11	Devolução	Confirmar devolução de empréstimo ativo	Status muda para "devolvido"	Funcional
CT12	Reserva	Selecionar usuário + livro	Reserva criada e listada	Funcional
CT13	Cancelamento de Reserva	Confirmar cancelamento	Status muda para "cancelada"	Funcional
CT14	Multa	Clicar em "pagar" e confirmar	Multa marcada como paga	Funcional
CT15	Tema escuro	Clicar no botão de tema	Interface alterna claro/escuro	Usabilidade
CT16	Pesquisa/Filtro	Digitar termo na busca de Livros	Lista filtrada pelo termo	Funcional

ID	Funcionalidade	Entrada	Resultado Esperado	Tipo
CT17	Controle de acesso	Acessar `/usuarios` direto pela URL sem login	**Esperado:** redirecionar para login	Segurança
CT18	Autenticação	Login com `admin@biblioteca.com` / senha errada	**Esperado:** "Credenciais inválidas"	Segurança
CT19	SQL Injection / XSS	Inserir `'; DROP TABLE usuarios; --` e `<script>` no nome	Entrada tratada como texto; sem quebra	Segurança
CT20	Responsividade	Abrir em viewport mobile (375px)	Layout adapta sem quebrar	Usabilidade

---

PARTE 4 — Execução dos Testes

> Execução automatizada (navegador headless) com o ambiente local do Anexo A. Os prints estão na pasta `evidencias/`.

Caso	Resultado	Evidência (print)
CT01	aprovado	`evidencias/CT01_dashboard.png`
CT02	aprovado	`evidencias/CT02_usuario_criado.png`
CT03	aprovado	`evidencias/CT03_email_invalido.png`
CT04	aprovado	`evidencias/CT04_nome_obrigatorio.png`
CT05	aprovado	`evidencias/CT05_editar_usuario.png`
CT06	aprovado	`evidencias/CT06_excluir_usuario.png`
CT07	aprovado (ver BUG08)	`evidencias/CT07_livro_criado.png`
CT08	aprovado	`evidencias/CT08_titulo_obrigatorio.png`
CT09	aprovado	`evidencias/CT09_copias_negativas.png`
CT10	aprovado	`evidencias/CT10_emprestimo_criado.png`
CT11	aprovado	`evidencias/CT11_devolucao.png`
CT12	aprovado	`evidencias/CT12_reserva_criada.png`
CT13	aprovado	`evidencias/CT13_reserva_cancelada.png`
CT14	aprovado	`evidencias/CT14_multas.png`
CT15	aprovado	`evidencias/CT15_tema_escuro.png`
CT16	aprovado	`evidencias/CT16_busca_filtro.png`
CT17	**reprovado (BUG01)**	`evidencias/CT17_acesso_sem_login.png`
CT18	**reprovado (BUG02)**	`evidencias/CT18_sem_tela_login.png`
CT19	aprovado	`evidencias/CT19_sqli_xss.png`
CT20	aprovado	`evidencias/CT20_responsivo_mobile.png`

Observações da execução:

- **CT03 / CT09:** o bloqueio de e-mail inválido e de cópias negativas ocorre pela validação **nativa do HTML5** (`type="email"` e `min="0"`), que dispara antes da mensagem customizada do React Hook Form. O efeito (entrada inválida impedida) é o esperado.
- **CT07:** o cadastro só foi concluído **preenchendo a Data de Publicação**. Com o campo de data vazio, o cadastro falha (ver **BUG08**).
- **CT10:** só foi possível registrar o empréstimo após cadastrar um usuário com cargo **"membro"** — o seed inicial não traz nenhum (ver **BUG09**).
- **CT12:** a reserva só é aceita para livro **sem cópias disponíveis** (regra de negócio correta); para livro disponível o sistema exibe "Livro disponível para empréstimo, não necessita reserva".
- **CT14:** não há multas no seed (multas dependem de empréstimos em atraso); o print registra a tela de multas vazia.
- **CT19:** a injeção `'; DROP TABLE usuarios; --` foi **armazenada como texto literal** e a tabela permaneceu intacta — queries parametrizadas no backend protegeram os dados.

PARTE 5 — Relatório de Bugs

Bug	Descrição	Severidade
BUG01	**Rotas desprotegidas:** não há `ProtectedRoute` nem tela de login no roteamento (`App.jsx`). Qualquer pessoa acessa `/usuarios` , `/livros` , `/emprestimos` etc. diretamente pela URL, sem autenticação.	**Alta**
BUG02	**Login hardcoded no frontend:** `AuthContext` valida credenciais fixas (`admin@biblioteca.com` / `admin123`) e grava um token falso (`fake-jwt-token`) no `localStorage` , sem chamar o backend. Não há validação real de identidade.	**Alta**
BUG03	**Sessão forjável:** qualquer valor em `localStorage.token` faz o app considerar o usuário como "Administrador" (o `useEffect` do `AuthContext` não valida o token).	**Alta**
BUG04	**Sem sanitização explícita de entrada:** campos como nome/título aceitam `<script>` e payloads de SQL injection; embora a UI trate como texto, não há limpeza/escape explícito no frontend.	**Média**
BUG05	**URL de API padrão aponta para produção:** sem a variável `VITE_API_URL` , o frontend conecta em `https://biblioteca-gateway.onrender.com` , dificultando testes locais e podendo expor o ambiente errado.	**Média**
BUG06	**ISBN sem validação de formato:** o campo ISBN aceita qualquer texto (sem máscara/checagem de dígitos).	**Baixa**
BUG07	**Mensagens de erro de API pouco amigáveis:** quando o backend está indisponível, a tela pode ficar vazia/sem feedback claro além de toast genérico.	**Baixa**
BUG08	**Cadastro de livro quebra com Data de Publicação vazia:** ao deixar a data em branco, o frontend envia `publicado_em: ""` (string vazia) e o backend falha com `invalid input syntax for type date: ""` . O usuário só vê um toast genérico ("o serviço de destino retornou um erro"), sem saber a causa. **Reproduzido em CT07.**	**Média**
BUG09	**Fluxo de empréstimo inviável no estado inicial:** o formulário de empréstimo só lista usuários com cargo "membro", mas o seed traz apenas um "bibliotecário". Sem cadastrar um membro manualmente, a funcionalidade principal (empréstimo) fica inutilizável.	**Média**
BUG10	**Lista de reservas não exibe nome do usuário/livro:** a tela de Reservas mostra "Sem e-mail" e não resolve o nome do usuário/título, dificultando a leitura.	**Baixa**

> **Observação de integração (backend):** durante a subida do ambiente local foram identificados dois defeitos de configuração no backend que impedem o stack de rodar "out of the box": (a) o **API Gateway** ignorava as variáveis de ambiente de URLs dos serviços (caindo em produção); (b) os serviços com banco entravam no modo "produção" por

> `NODE_ENV=production` e ignoravam `PGHOST`. Ambos foram contornados para viabilizar os testes (ver Anexo A) e estão descritos como observação, não como bug do frontend.

Critérios de Severidade (referência)

- **Alta:** quebra o sistema, perda de dados, falha de segurança ou funcionalidade principal inutilizável.
- **Média:** afeta funcionalidades, impacta a experiência, mas possui solução alternativa.
- **Baixa:** problemas visuais, pequenos erros de usabilidade, layout/alinhamento.

PARTE 6 — Análise Crítica

- **Quais testes faltaram durante o desenvolvimento?** Testes de autenticação/autorização e testes de integração frontend↔backend. A autenticação está apenas "simulada" e nunca foi validada de ponta a ponta.
- **Quais erros poderiam afetar usuários reais?** BUG01/BUG02/BUG03 (segurança): qualquer pessoa acessaria e alteraria dados sem login. BUG05 (apontar para produção) poderia fazer um teste alterar dados reais. BUG08 (data vazia) faria um bibliotecário não conseguir cadastrar livros sem entender o motivo. BUG09 impediria o uso da função central (empréstimo) logo após a instalação.
- **Quais partes precisam de automação?** Validação de formulários (Usuário/Livro), fluxo de empréstimo/devolução e testes de rota protegida — bons candidatos a testes E2E (Cypress/Playwright) e unitários (Vitest + Testing Library).
- **Qual área do sistema é mais frágil?** A **camada de autenticação/controle de acesso** do frontend — hoje é apenas decorativa (hardcoded, sem proteção de rotas), sendo o maior risco de segurança do sistema.

PARTE 7 — Testes Adicionais (Tentando Quebrar a Aplicação)

#	Teste	Procedimento	Resultado Esperado
T1	**Acesso sem autenticação**	Abrir `/usuarios`, `/livros` direto na URL	(Confirma BUG01) acessa sem login
T2	**Sessão forjada**	No console: `localStorage.setItem('token','x')` e recarregar	(Confirma BUG03) entra como Administrador
T3	**SQL Injection**	Nome do usuário = `'; DROP TABLE usuarios; --`	Tratado como texto; tabela intacta
T4	**XSS**	Título do livro = `<script>alert(1)</script>`	Renderizado como texto, sem executar script
T5	**Campos vazios**	Enviar formulários em branco	Mensagens de obrigatoriedade
T6	**Dados inválidos**	E-mail `aaa`, cópias `-5`, ISBN `abc`	E-mail/cópias bloqueados; ISBN aceito (BUG06)
T6b	**Data de publicação vazia**	Cadastrar livro sem preencher a data	(Confirma BUG08) cadastro falha com erro genérico
T7	**Requisições repetidas**	Clicar "Criar" várias vezes rápido	Verificar duplicação de registros
T8	**Backend offline**	Parar o gateway e navegar	Mensagem de erro/toast (verificar BUG07)
T9	**Responsividade**	Redimensionar para 320–375px	Layout adapta sem overflow
T10	**Teste de carga simples**	Disparar várias requisições à API	Verificar estabilidade/tempo de resposta

Anexo A — Ambiente de Testes (como reproduzir)

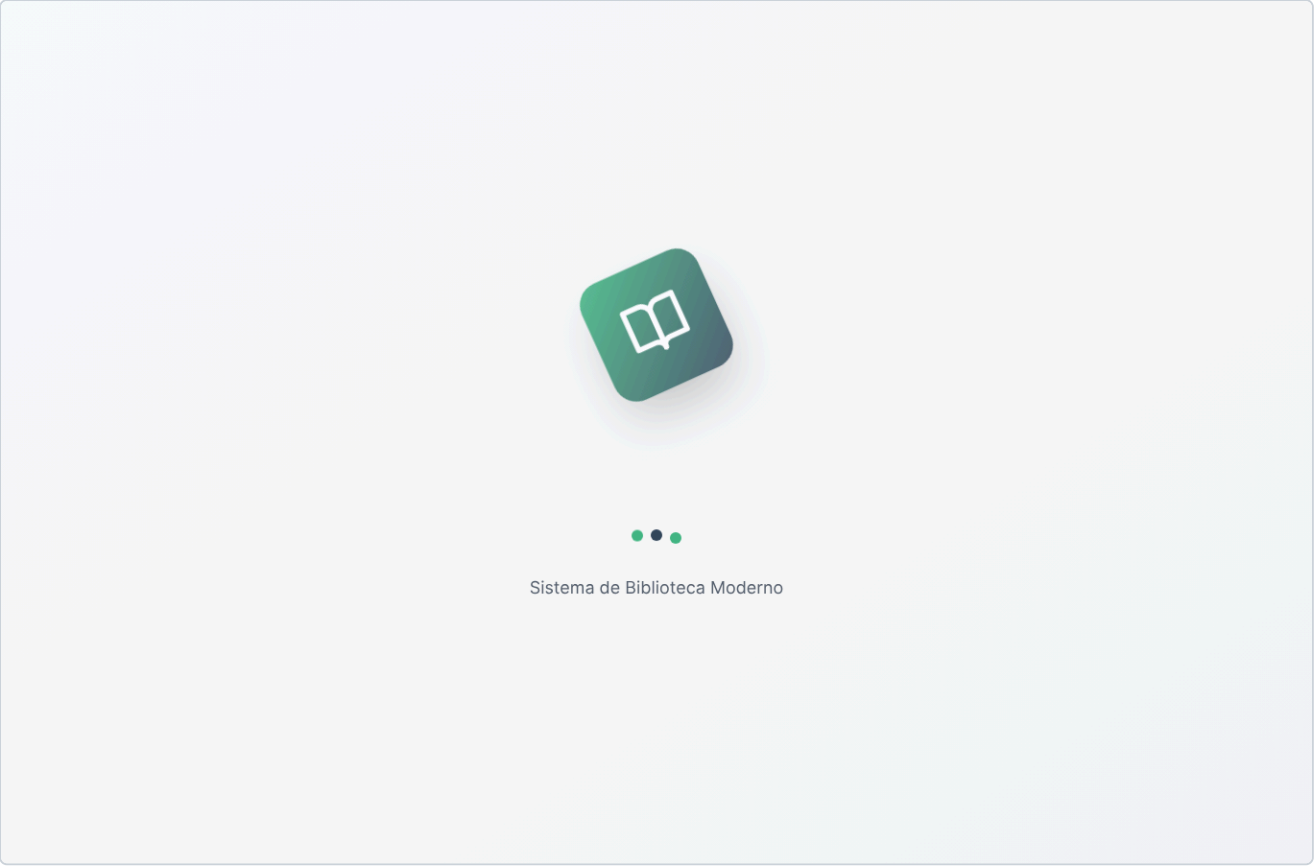
Frontend e backend foram executados **localmente**:

- **Frontend (React/Vite):** http://localhost:5180
  - `.env` com `VITE_API_URL=http://localhost:3100`
  - Comando: `npm install && npm run dev -- --port 5180 --host`
- **Backend (microsserviços via Docker):** API Gateway em http://localhost:3100
  - Comando: `docker compose up -d --build` (no repositório `biblioteca-api`)
  - Portas remapeadas via `docker-compose.override.yml` para evitar conflitos locais (gateway `3100`, multas `3104`, postgres usuários `5442`, multas `5445`).
- **Dados de exemplo (seed):** usuário "Admin Biblioteca" e livros (Clean Code, Domain-Driven Design, etc.); empréstimos/reservas/multas iniciam vazios.

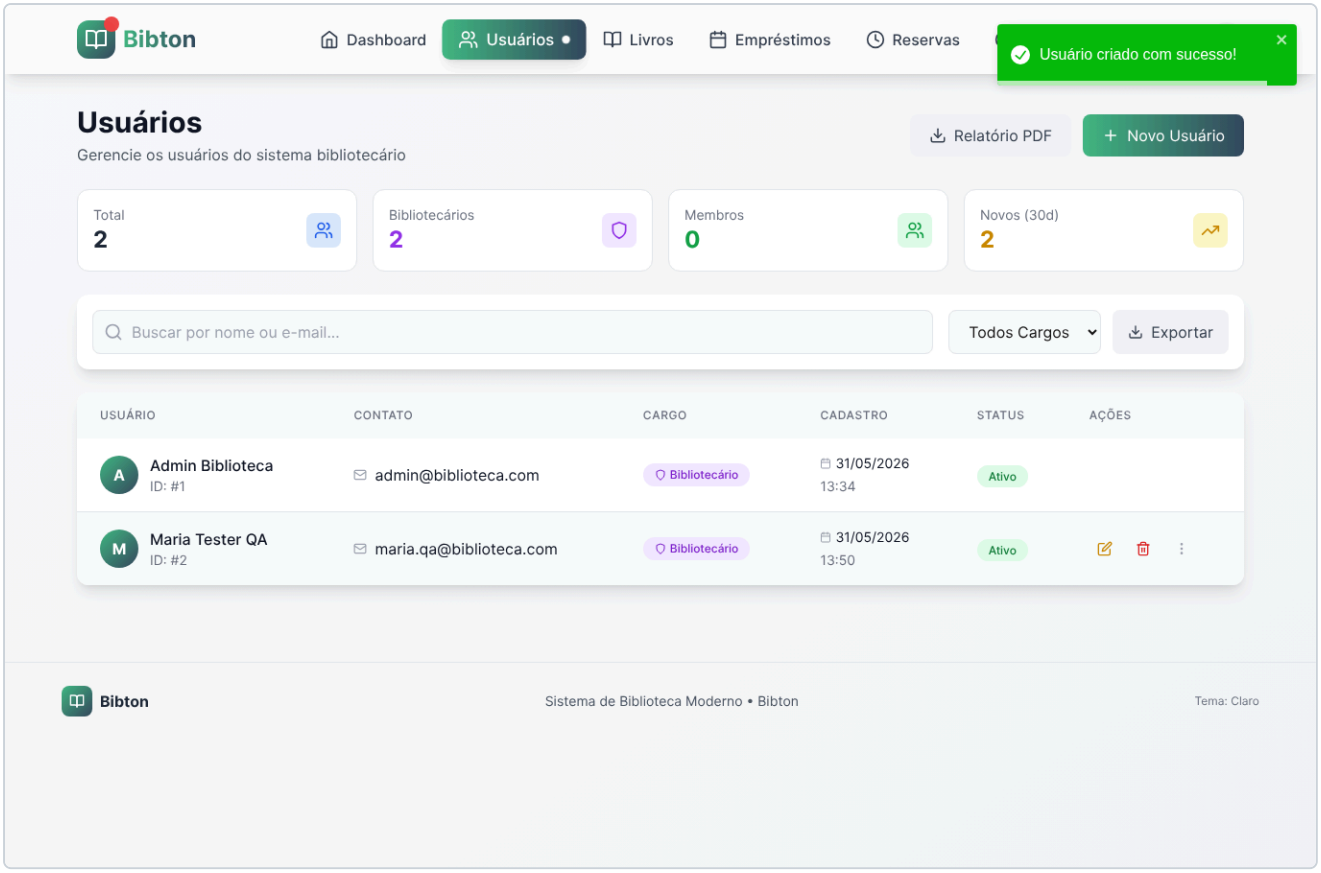
> **Login simulado (frontend):** `admin@biblioteca.com` / `admin123` — ver BUG02.

Anexo B — Evidências (Prints)

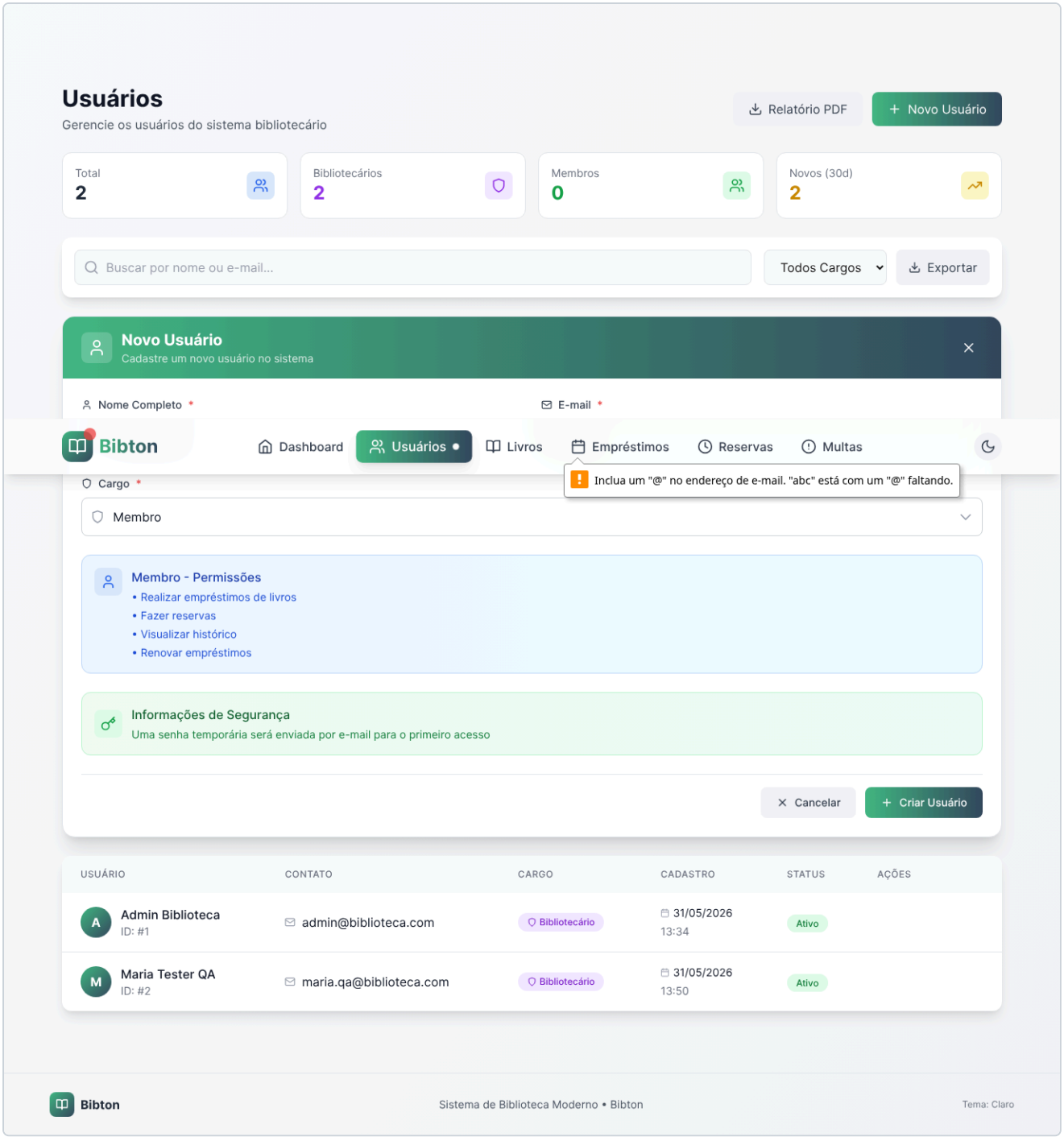
CT01 — Dashboard carregado com dados da API



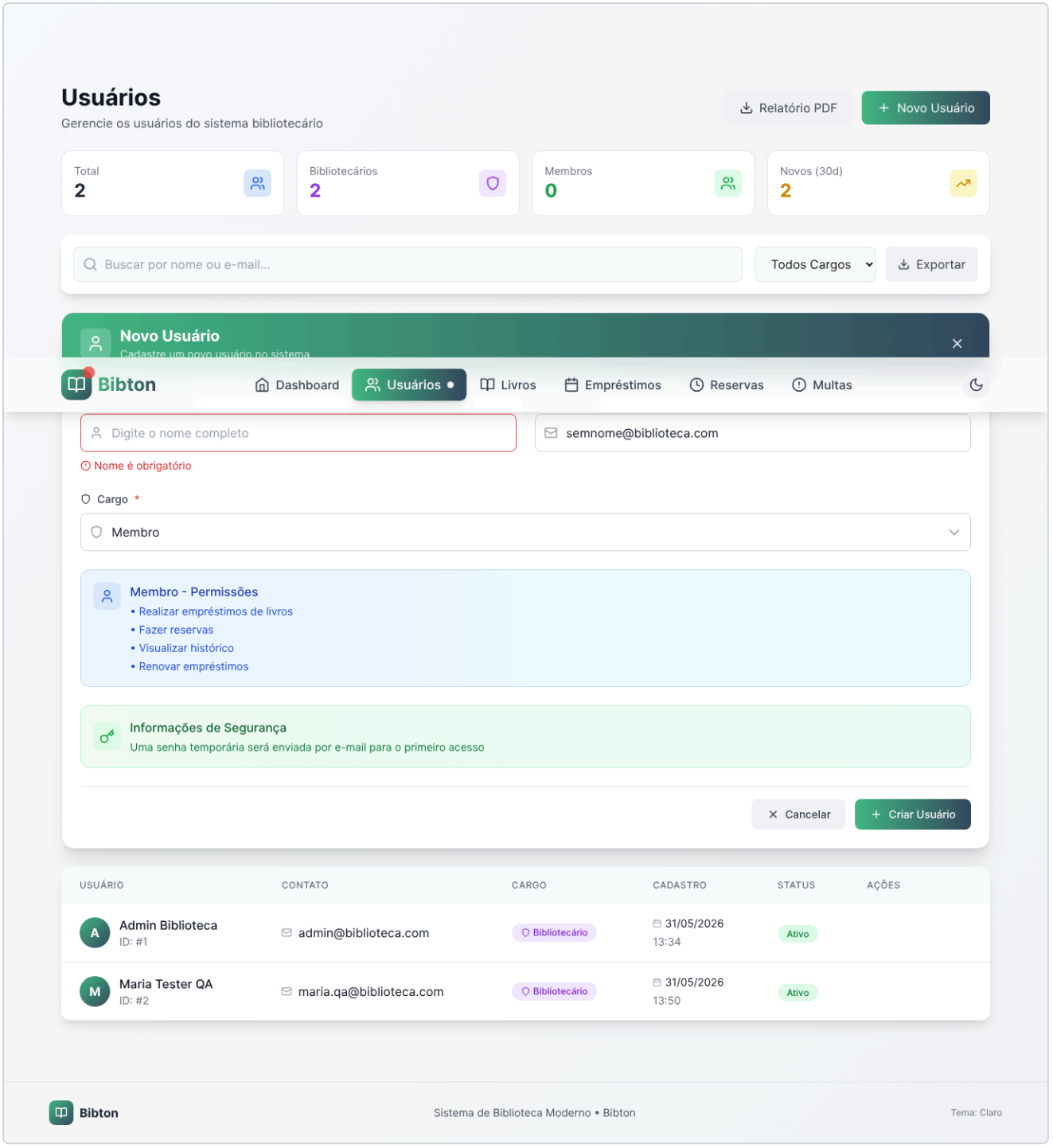
CT02 — Usuário válido criado e listado



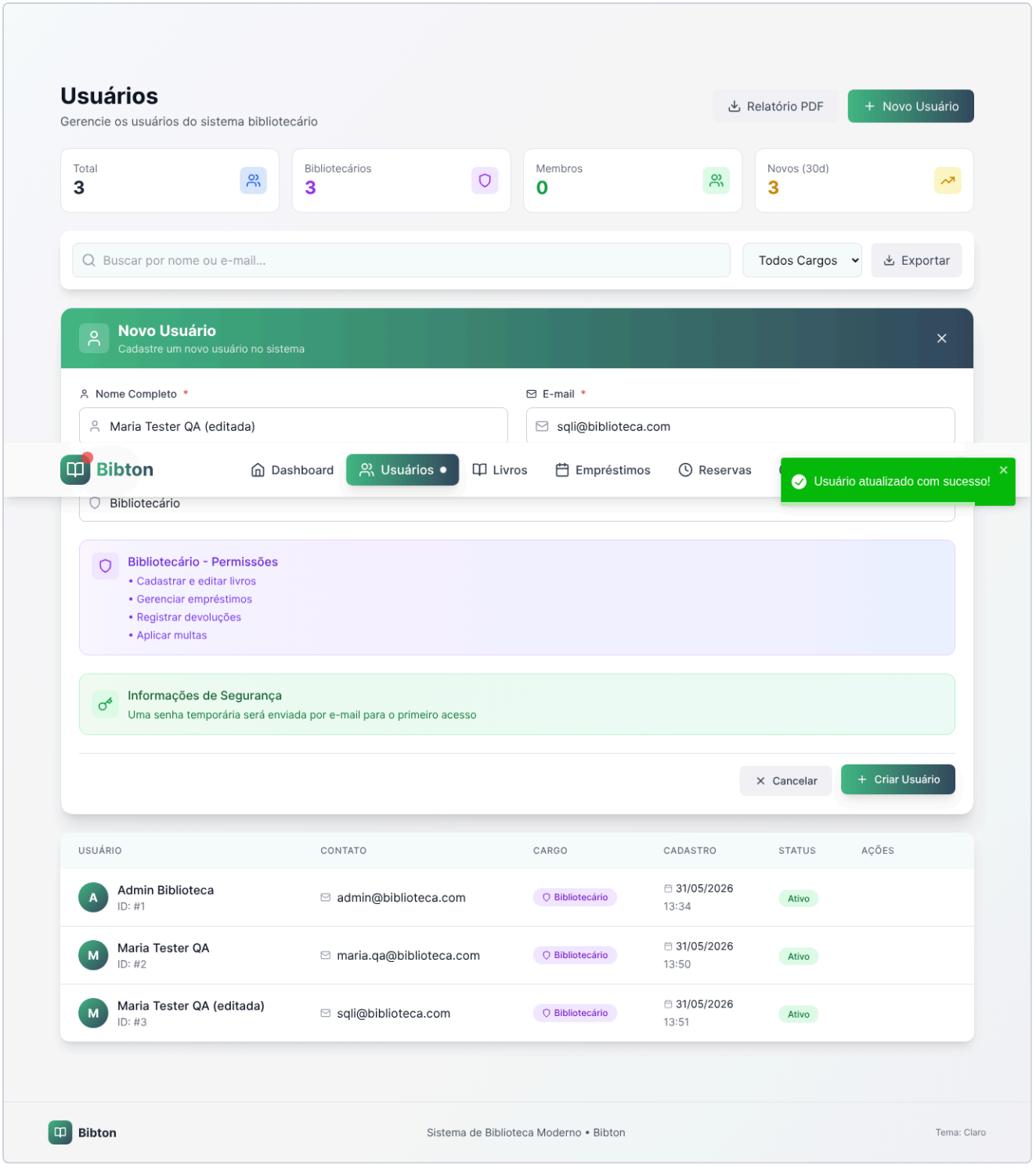
CT03 — E-mail inválido bloqueado (validação)



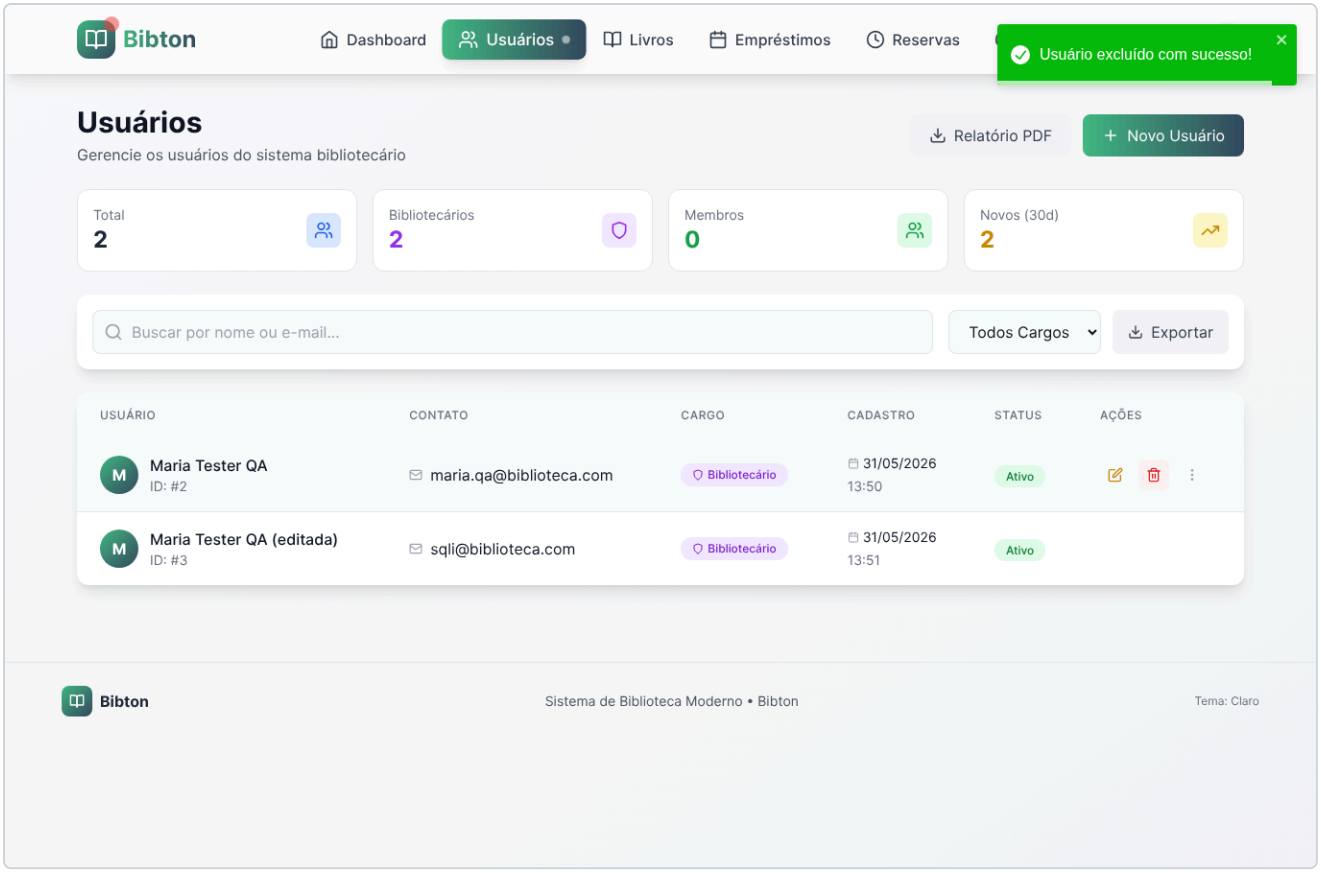
CT04 — Nome obrigatório bloqueia cadastro



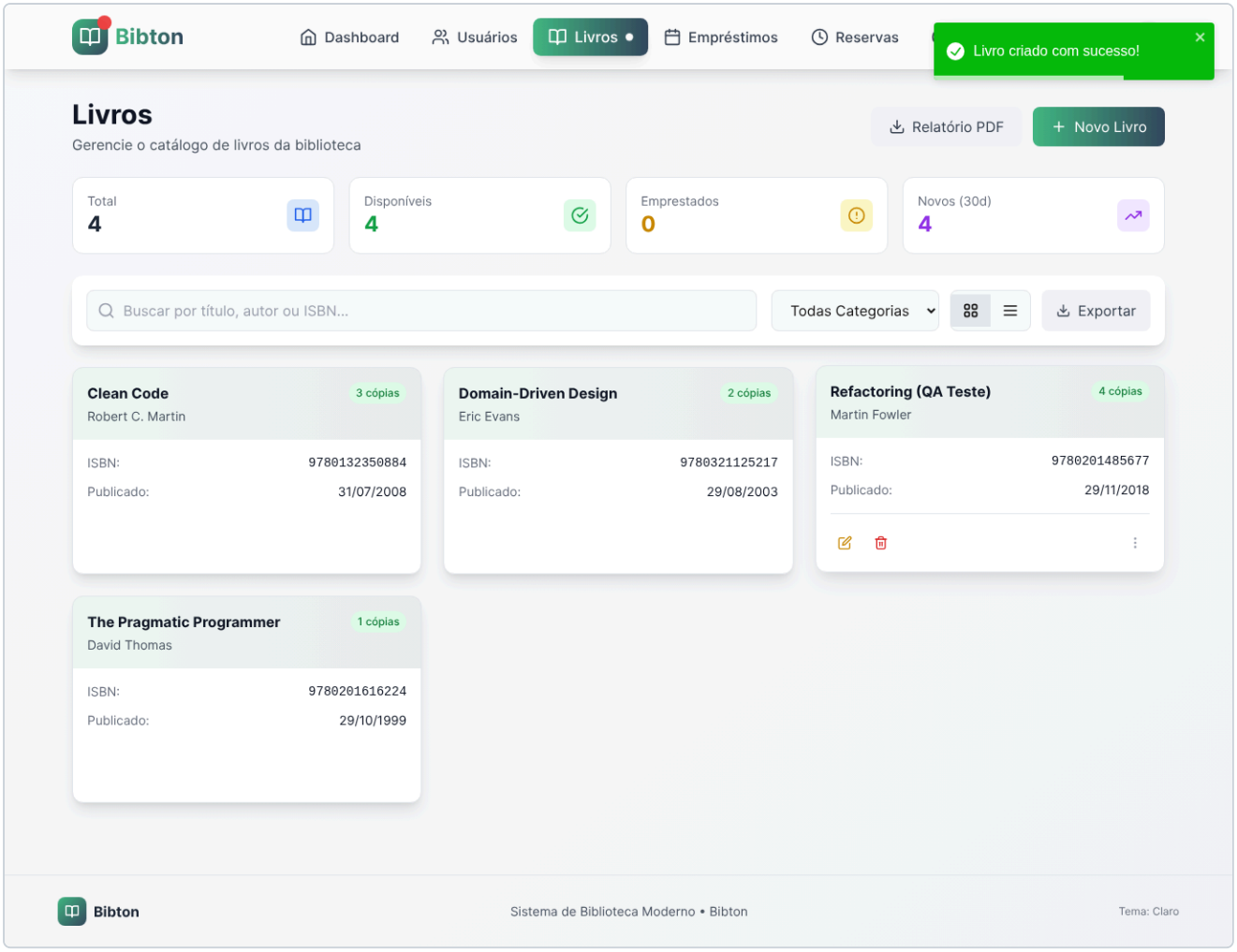
CT05 — Edição de usuário



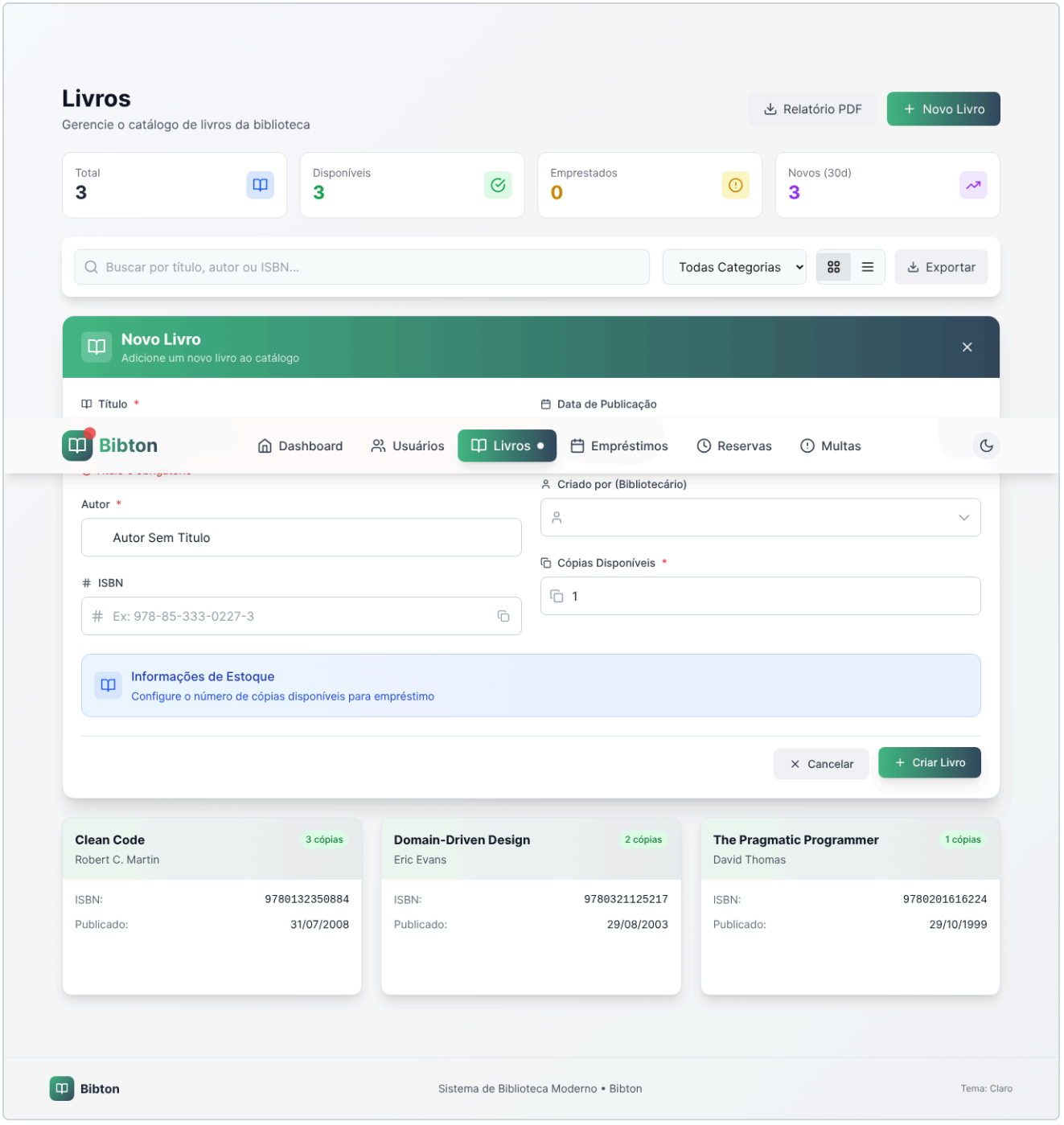
CT06 — Exclusão de usuário



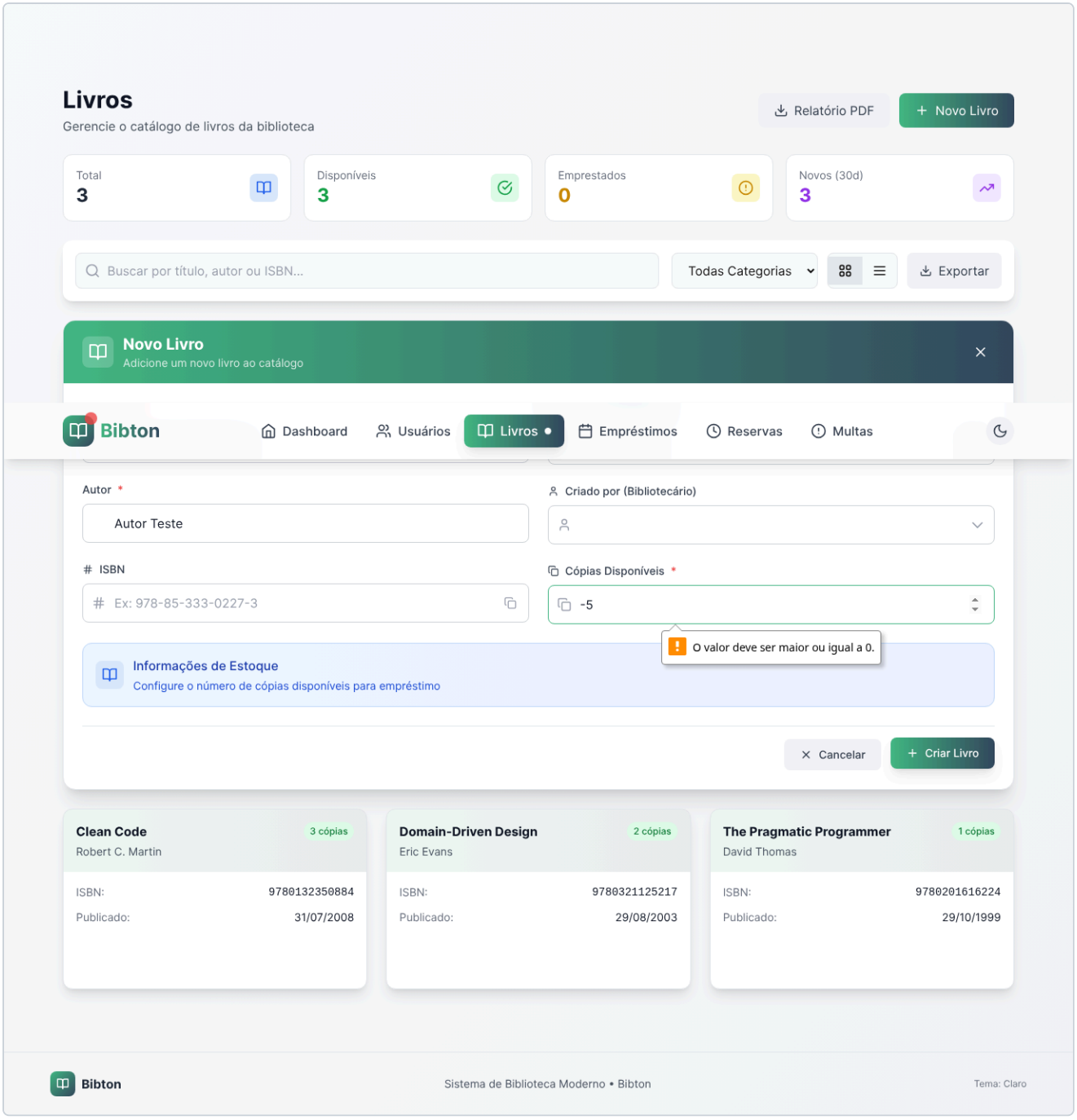
CT07 — Livro criado com sucesso



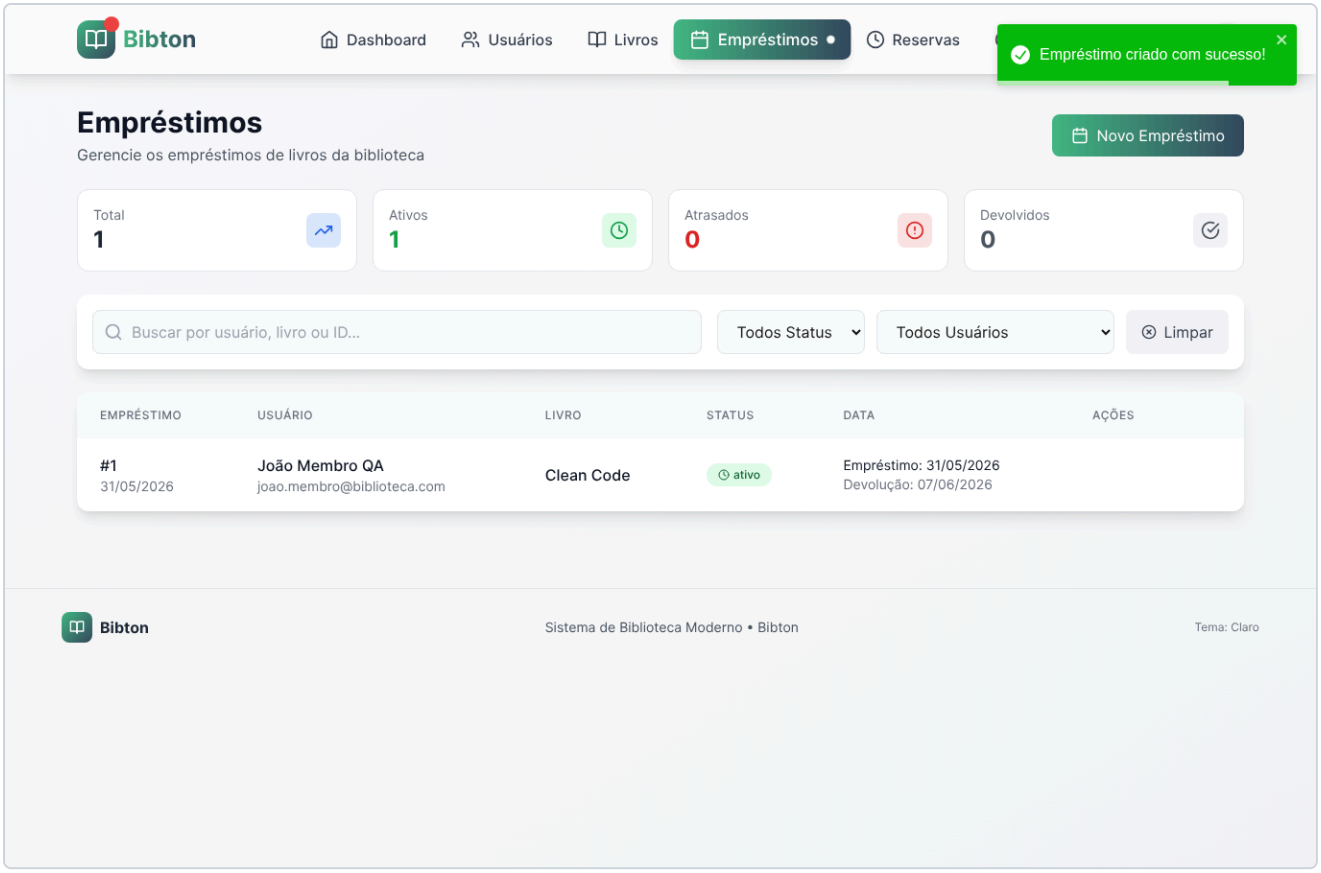
CT08 — Título obrigatório bloqueia cadastro



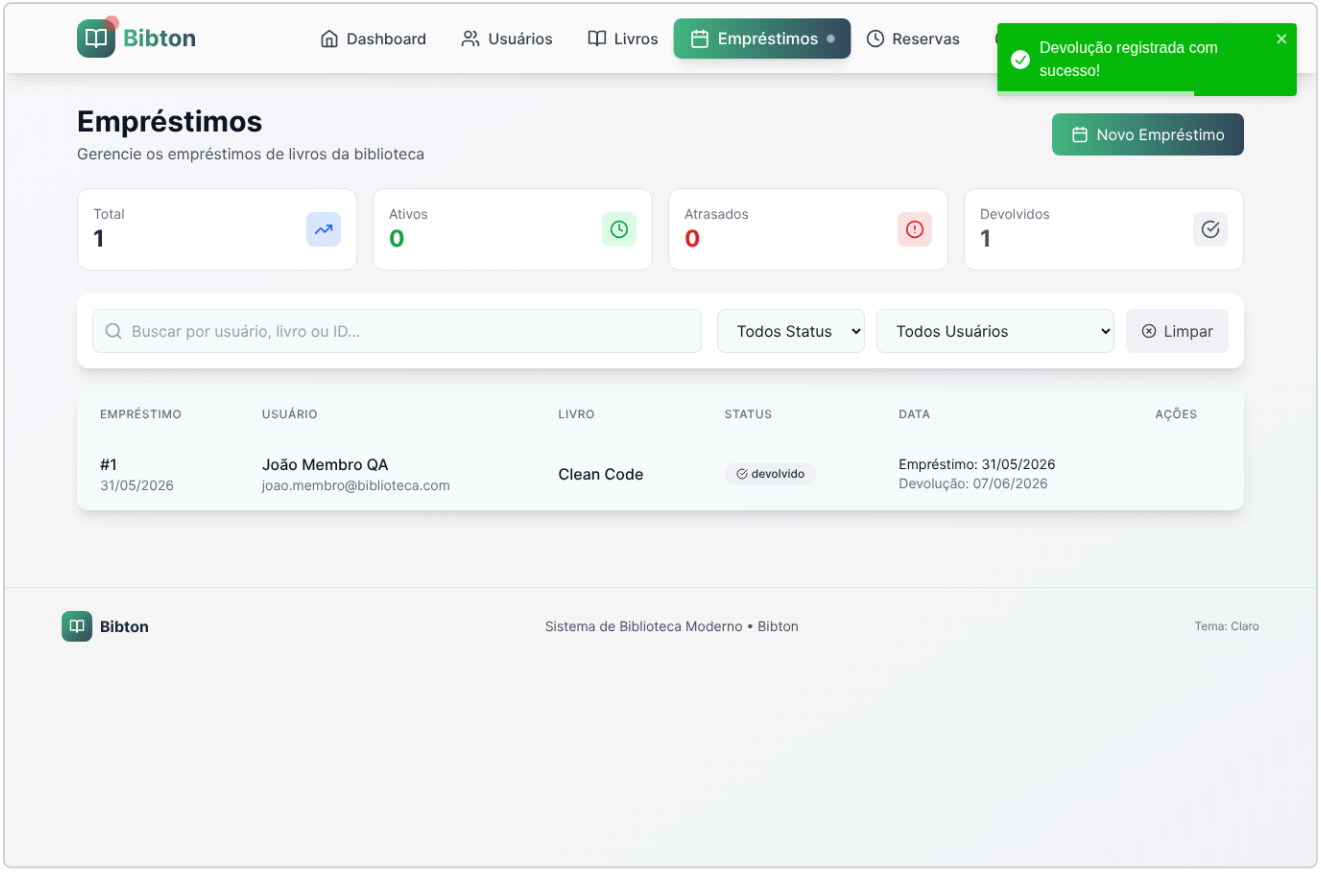
CT09 — Cópias negativas bloqueadas (validação)



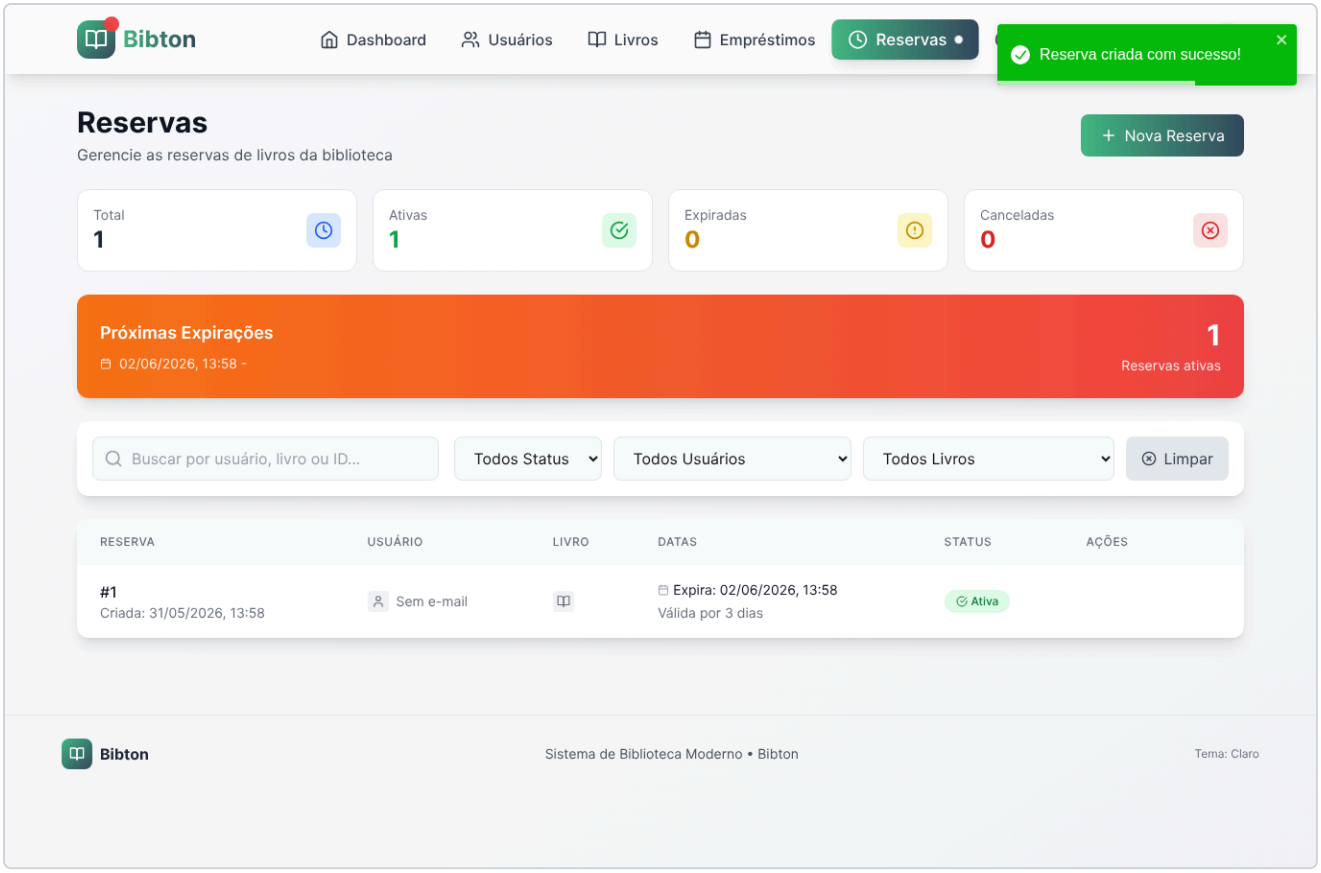
CT10 — Empréstimo registrado



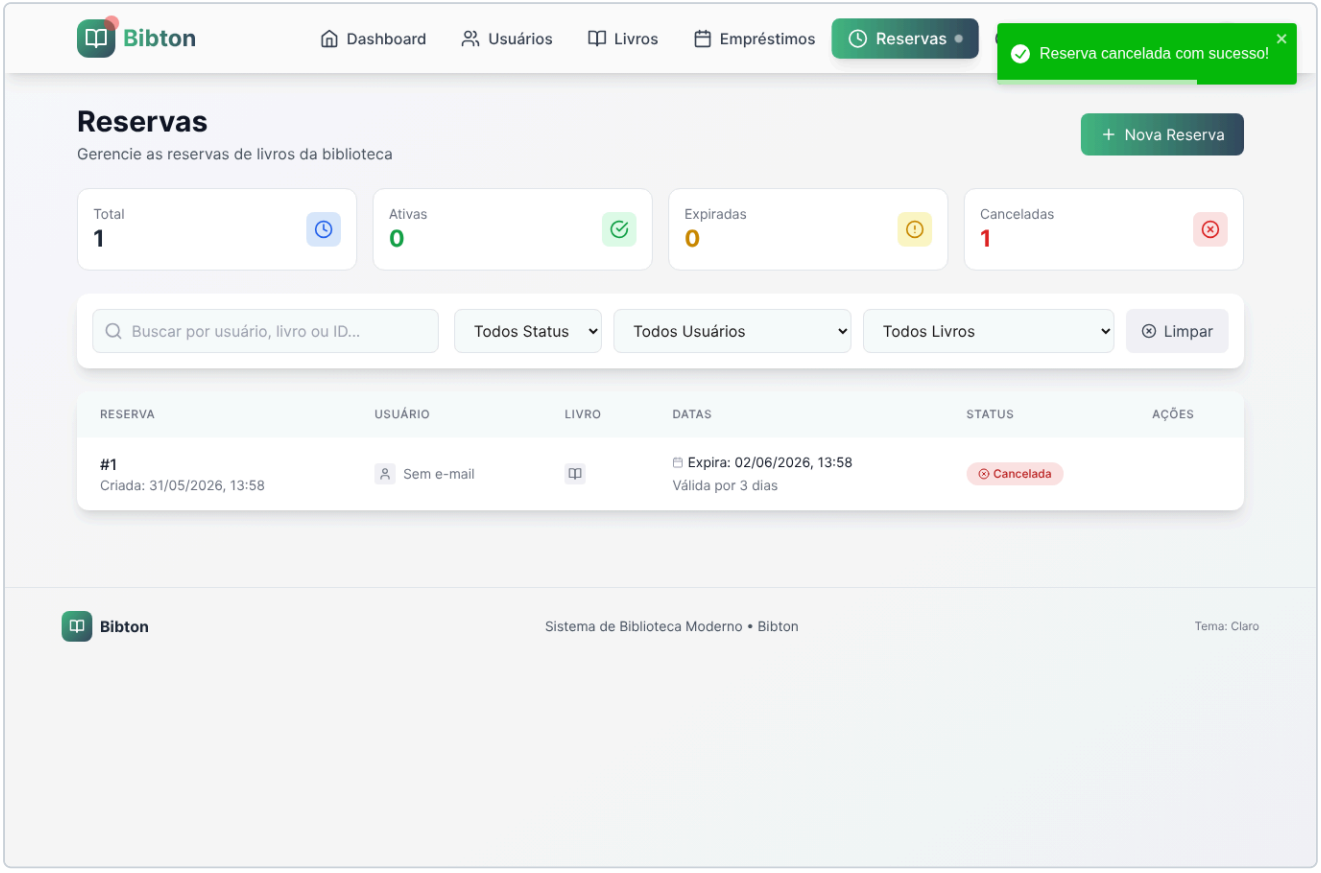
CT11 — Devolução registrada



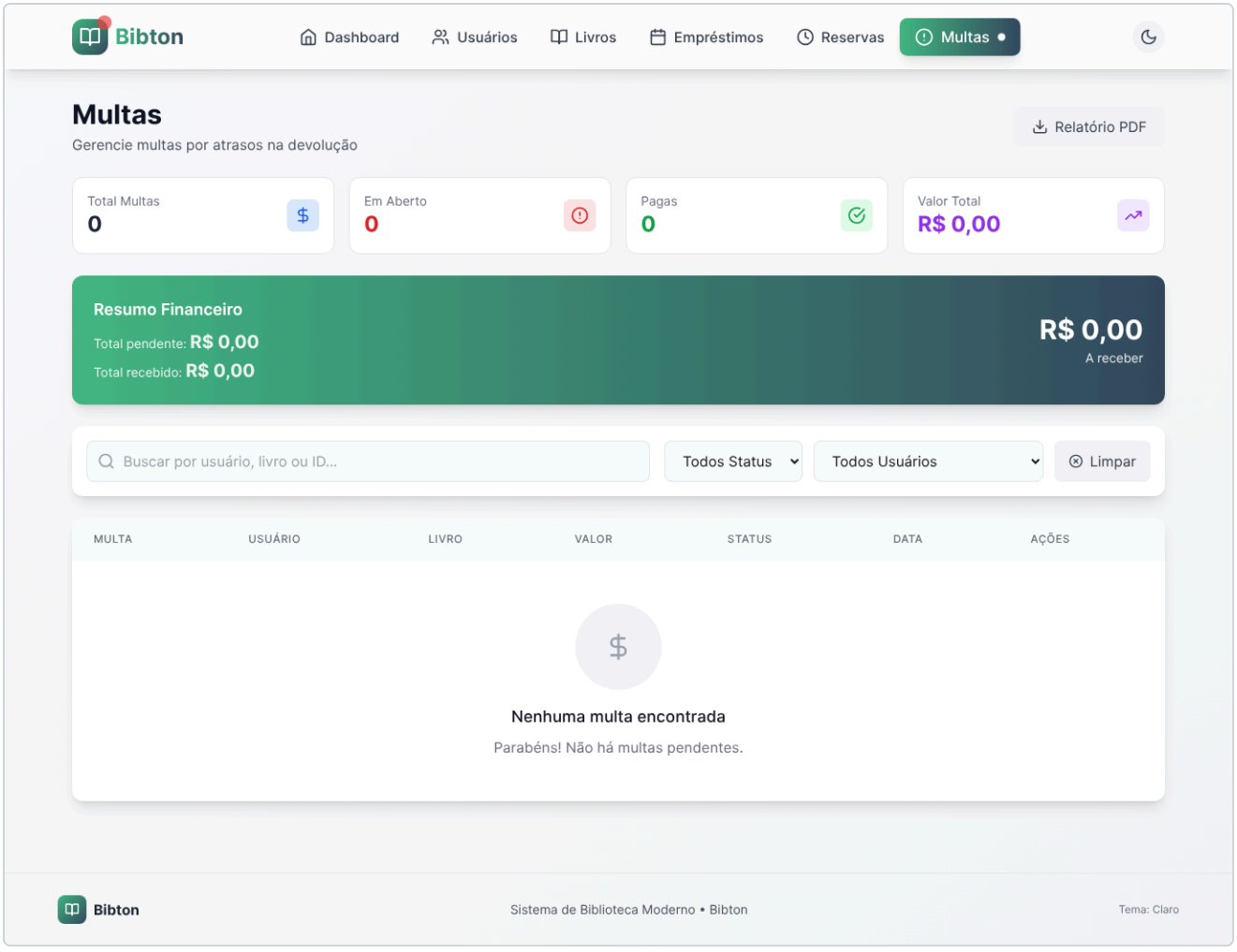
CT12 — Reserva criada com sucesso



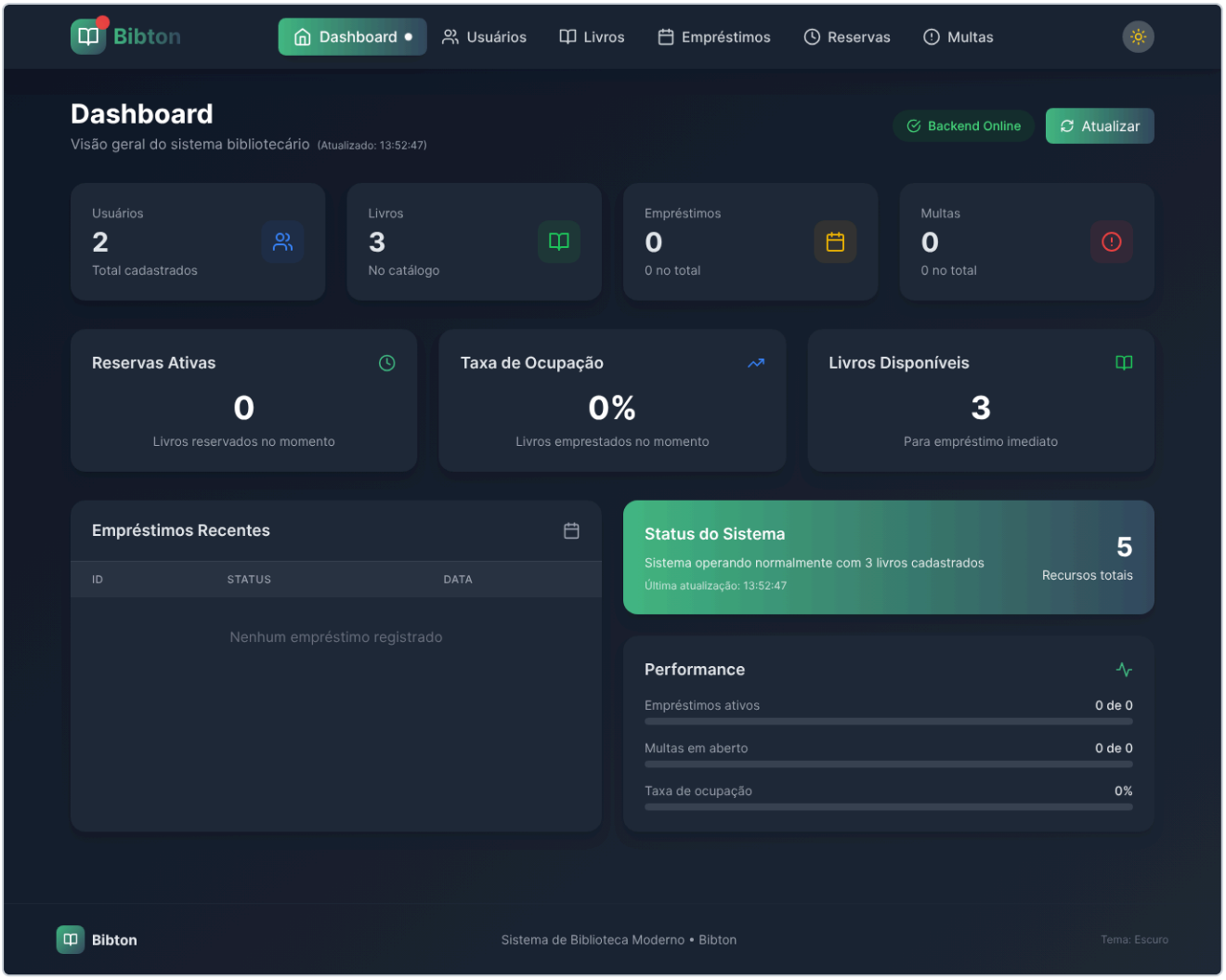
CT13 — Reserva cancelada



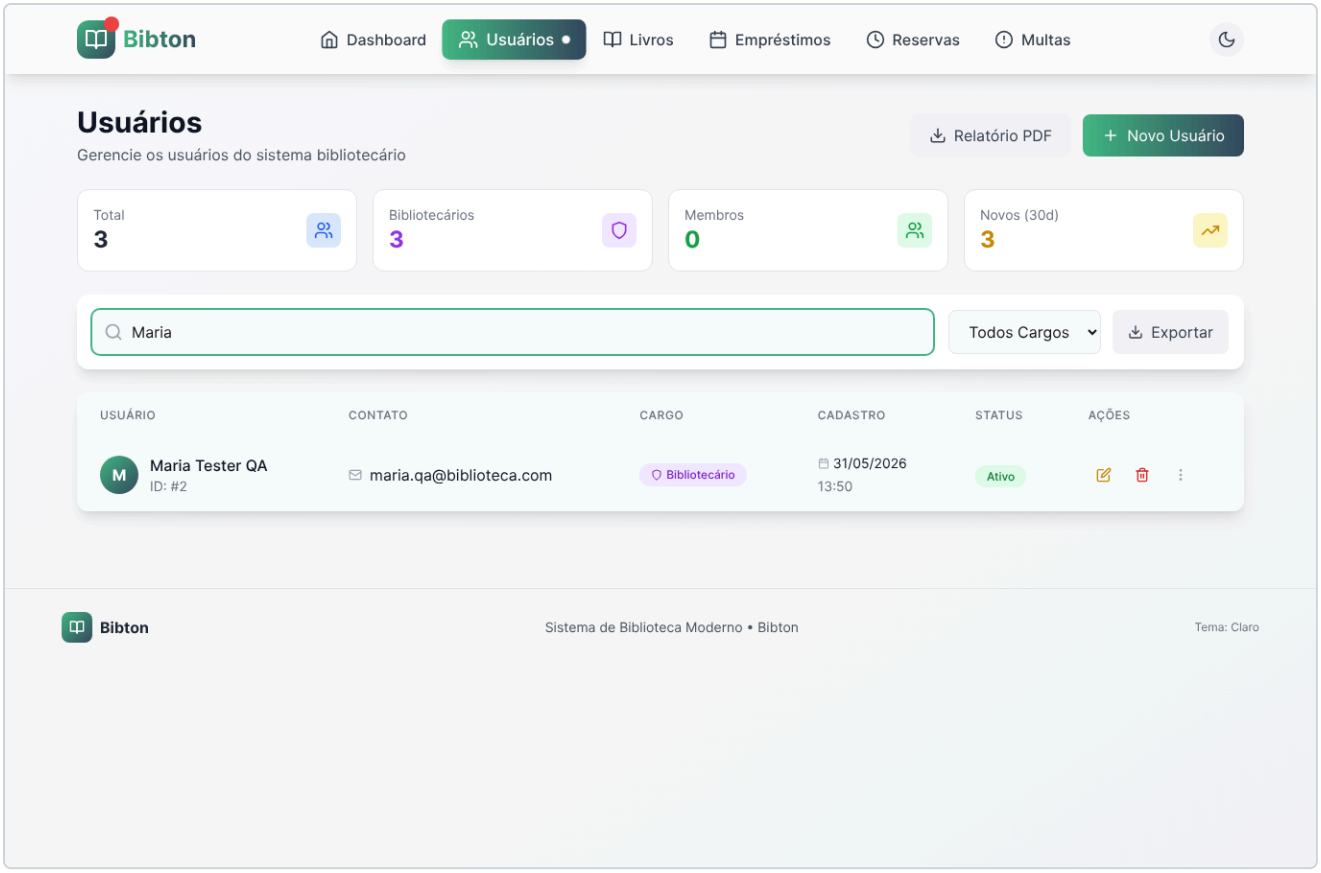
CT14 — Tela de Multas



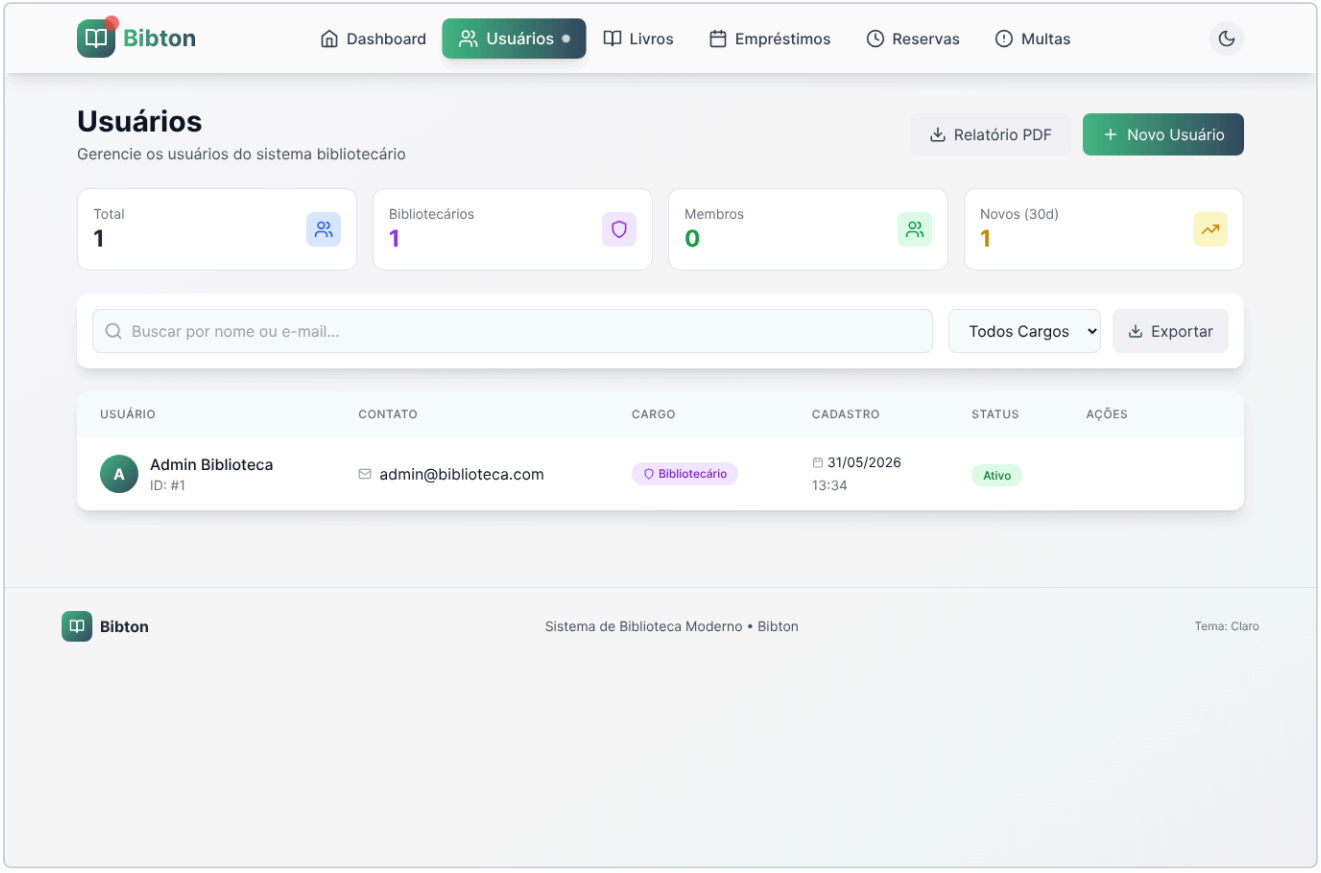
CT15 — Tema escuro (dark mode)



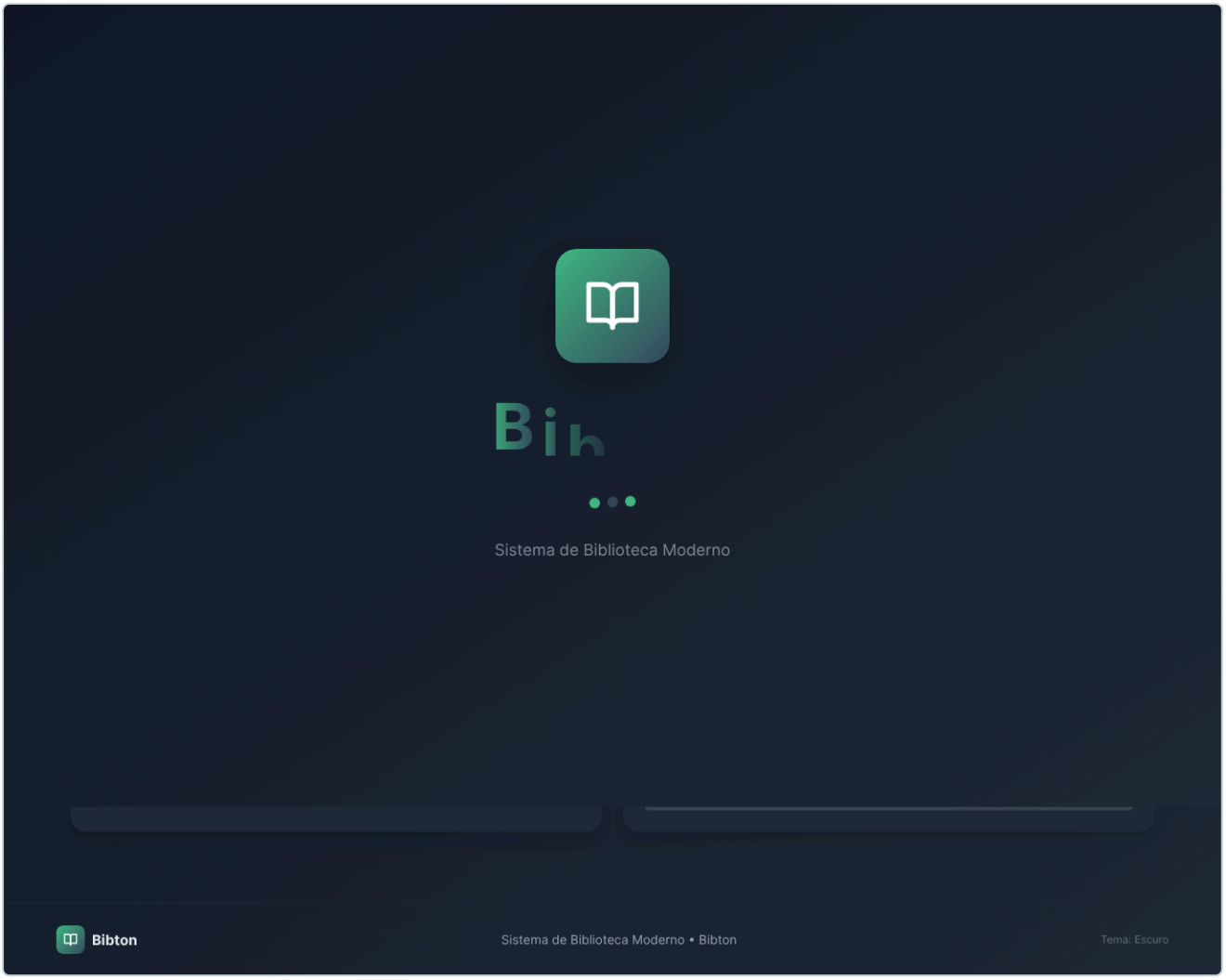
CT16 — Pesquisa / filtro



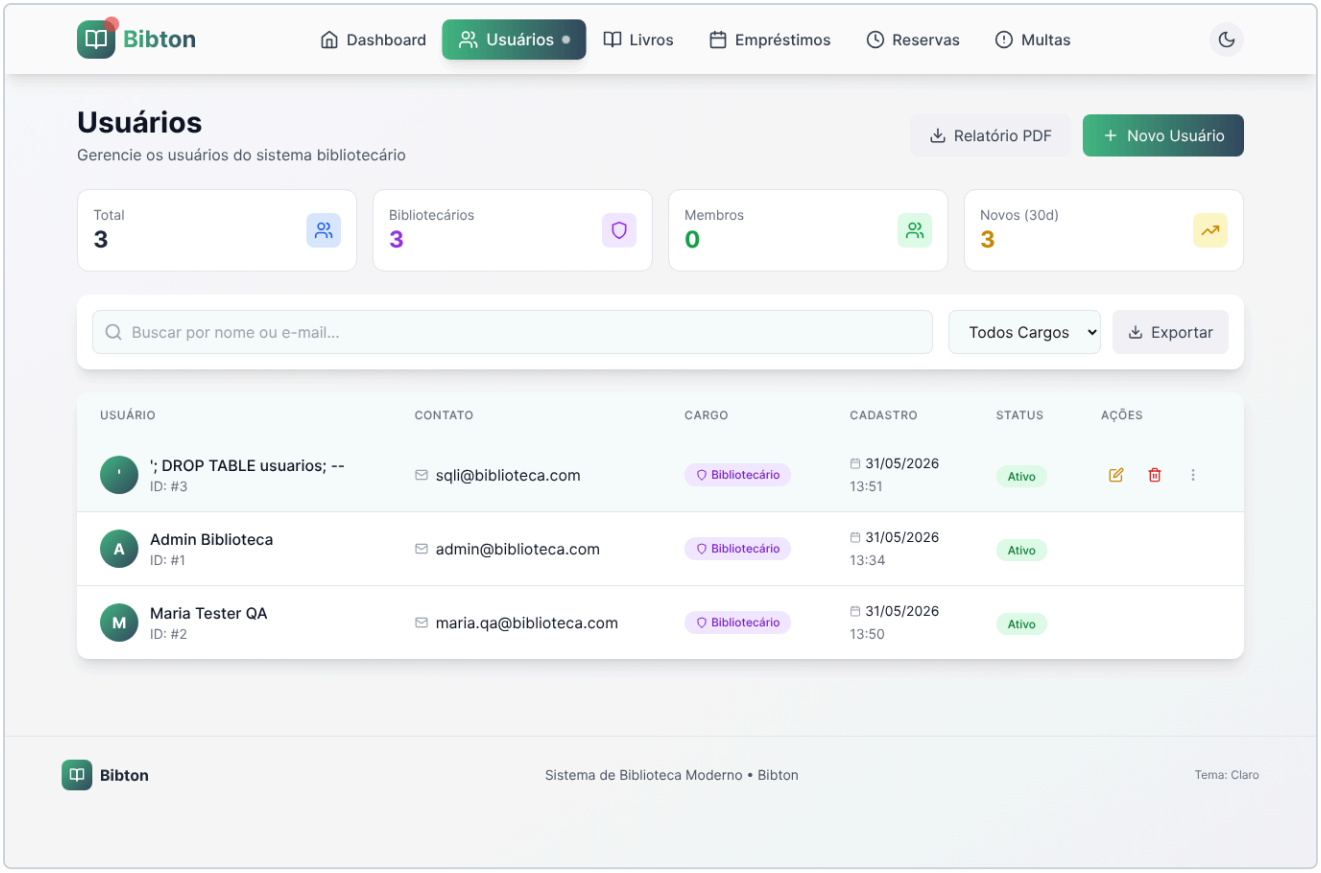
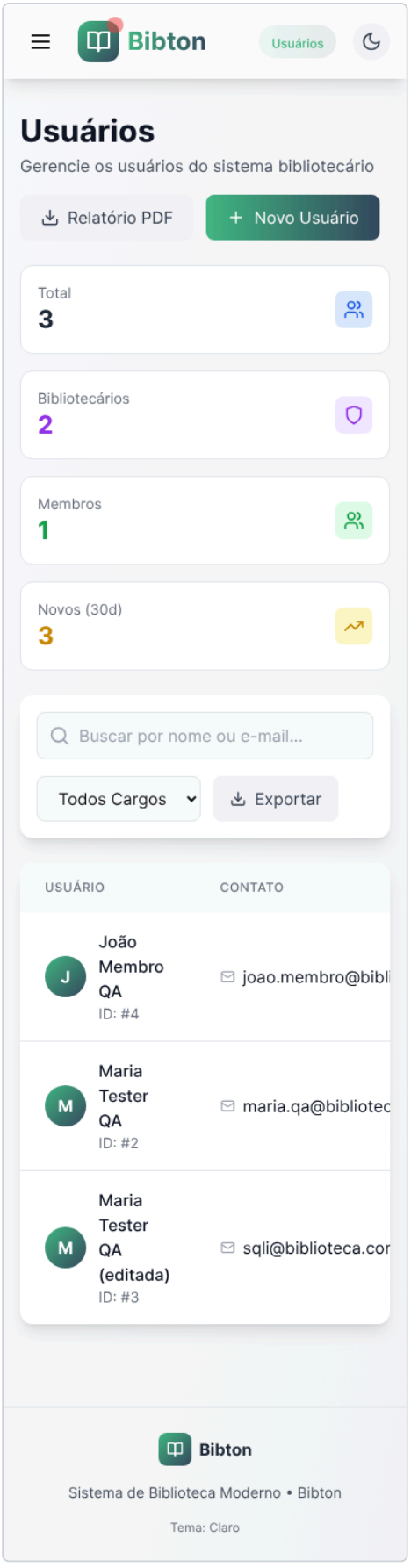
CT17 — BUG01 — acesso a /usuarios sem login



CT18 — BUG02 — /login redireciona (não há tela de login)



CT19 — SQL Injection tratado como texto; tabela intacta



CT20 — Responsividade mobile (390px)